École Nationale Supérieure des Arts et Métiers
ENSAM Casablanca

Comparative Analysis of Stemming Techniques in Text Mining

Mini-Project Report — Natural Language Processing

Authors: Taha MAHHA Malak MEKYASSI
Institution: ENSAM Casablanca
Date: April 2026

Objective: How do Porter, Snowball, and Lancaster stemmers affect the performance and efficiency of a Random Forest classifier across binary and multi-class NLP tasks?

Abstract

Stemming is a fundamental normalisation step in the Natural Language Processing (NLP) pipeline that reduces inflected or derived word forms to their morphological root. This report presents a systematic comparison of three widely adopted stemming algorithms — Porter, Snowball (English), and Lancaster — against a non-stemmed baseline across two distinct text-classification benchmarks: a binary Fake-News detection task and a multi-class technical documentation categorisation task. Each configuration is evaluated inside a leakage-free `sklearn.Pipeline` using cross-validated F1-Macro scores, processing latency, and TF-IDF vocabulary size as primary metrics. Results indicate that for binary classification, no stemming achieves peak accuracy (F1: 0.9915). However, in multi-class settings, the Porter stemmer provides the highest predictive performance (F1: 0.9683), while Snowball remains the optimal middle-ground choice for balancing stemming logic and computational speed. These findings yield actionable heuristics for deploying text-mining pipelines in production environments.

Keywords: stemming, text normalisation, TF-IDF, Random Forest, F1-Macro, fake news detection, text classification.

Contents

1 Introduction **3**

2 Background Concepts **3**
2.1 Stemmer 3
2.2 Random Forest Classifier 3
2.3 Confusion Matrix 3

3 Dataset Presentation **4**
3.1 Dataset 1 — Fake News Detection 4
3.2 Dataset 2 — Text Classification Documentation 4

4 Methodology **4**
4.1 Stemming Algorithms Used 4
4.2 Feature Extraction (TF-IDF) 5
4.3 Evaluation Metrics 5
4.4 Libraries Utilised 5
4.5 Time Complexity of the Pipeline 6

5 Results and Performance Analysis **6**
5.1 F1-Macro Scores 6
5.2 Confusion Matrices 6
5.3 Quantitative Comparison Table 7
5.4 Discussion 8

6 Model Interpretability: Feature Importance **8**

7 Conclusion **9**

1. Introduction

The preprocessing stage of any NLP pipeline is decisive in determining the quality of the representations fed to downstream models. Among the many preprocessing operations available, stemming occupies a central role. This study addresses the following research question:

> *How do Porter, Snowball, and Lancaster stemming algorithms affect the classification performance and computational efficiency of a Random Forest text-classifier, compared with a non-stemmed baseline, across a binary and a multi-class NLP task?*

The remainder of the report is structured as follows. Section 2 defines core concepts. Section 3 describes the datasets. Section 4 details the methodology and mathematical formulations. Section 5 presents quantitative results. Section 6 provides model interpretability analysis, and Section 7 concludes with recommendations.

2. Background Concepts

To contextualise the experimental setup, we define the foundational components of the text-mining pipeline evaluated in this study.

2.1. Stemmer

A **stemmer** is an algorithm designed to strip affixes (suffixes, prefixes) from words, reducing them to their base or root form (the "stem"). The goal is to ensure that related words map to the same conceptual feature. For instance, *running*, *runs*, and *ran* are reduced to the stem *run*. This achieves vocabulary normalisation and dimensionality reduction, which subsequently decreases the memory footprint of text representations.

2.2. Random Forest Classifier

A **Random Forest Classifier** is an ensemble learning method that constructs a multitude of decision trees at training time and outputs the mode of the classes (classification) of the individual trees. It operates by applying bagging (bootstrap aggregating) and random feature selection, which makes it robust against overfitting, particularly in high-dimensional and sparse feature spaces typical of TF-IDF text representations.

2.3. Confusion Matrix

A **Confusion Matrix** is a performance measurement tool for machine learning classification. It is a specific table layout that allows visualisation of the performance of an algorithm. Each row of the matrix represents the instances in a predicted class,

while each column represents the instances in an actual class. It reports the counts of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN).

3. Dataset Presentation

Two publicly available benchmark corpora were selected to cover complementary classification scenarios.

3.1. Dataset 1 — Fake News Detection

The first corpus is the *Fake News Detection Dataset*. Articles were labelled $y = 1$ for genuine content and $y = 0$ for fake content. A stratified random subsample was drawn to yield a balanced binary classification task.

3.2. Dataset 2 — Text Classification Documentation

The second corpus contains technical documentation snippets associated with multiple category labels, forming a multi-class classification problem.

Figure 1 visualises the class distribution across the datasets.

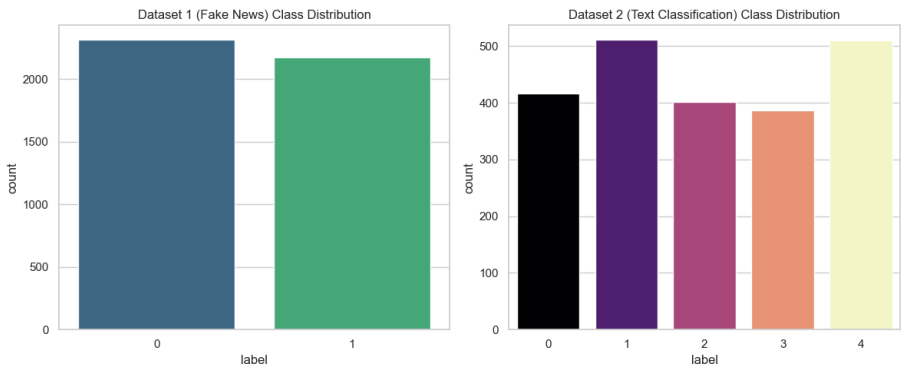

Figure 1: Class distributions for the Binary Fake-News Detection dataset and the Multi-class Documentation Classification dataset.

Both datasets were passed through a shared `backbone_cleaner` function that lowercases text and removes all non-alphabetic characters.

4. Methodology

4.1. Stemming Algorithms Used

Four experimental conditions were evaluated:

No Stemmer (Baseline). Only stopword removal is applied. This establishes the performance ceiling of the pipeline without morphological normalisation.

Porter Stemmer. A classic rule-based algorithm that applies a deterministic, multi-step sequence of suffix-stripping rules to English words. It is conservative and widely adopted but may generate non-linguistic stems.

Snowball Stemmer (English). A second-generation algorithm implemented within the Snowball framework. It corrects inconsistencies of the original Porter algorithm, offering slightly faster and more precise logical stemming.

Lancaster Stemmer. The most aggressive algorithm tested. It uses a heavily iterative suffix-stripping rule set without strict stopping criteria. This results in the highest vocabulary compression but suffers from significant over-stemming.

4.2. Feature Extraction (TF-IDF)

All text representations were constructed using **TF-IDF vectorisation**. The TF-IDF weight of term t in document d is mathematically defined as:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log\left(\frac{N}{1 + \text{df}(t)}\right) \tag{1}$$

where N is the total number of documents and $\text{df}(t)$ is the number of documents containing term t. The feature space was bounded to a maximum of 5,000 features.

4.3. Evaluation Metrics

To evaluate the classification models, we utilised the **F1-Macro Score**. This is the harmonic mean of per-class Precision and Recall, averaged with equal weight across all classes:

$$F_1^{\text{macro}} = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \frac{2 \cdot P_c \cdot R_c}{P_c + R_c} \tag{2}$$

Macro-averaging ensures that minority classes contribute equally to the final score, avoiding the overoptimism of accuracy in imbalanced settings.

4.4. Libraries Utilised

The implementation relies on the following standard Python ecosystem:

- **Pandas**: For efficient dataset loading, parsing, and data-frame manipulations.
- **NLTK (Natural Language Toolkit)**: Utilised for stopword corpora and accessing the Porter, Snowball, and Lancaster stemmer implementations.

- **Scikit-learn**: Employed for building the end-to-end `Pipeline`, specifically the `TfidfVectorizer`, `RandomForestClassifier`, and cross-validation utilities.
- **Matplotlib & Seaborn**: For rendering feature importance graphs, F1-score comparisons, and confusion matrices.

4.5. Time Complexity of the Pipeline

The time complexity of the pipeline is governed by two main phases: text preprocessing and classifier training. The application of a stemmer operates in $O(N \cdot L)$ time, where N is the total number of words in the corpus and L is the average word length. Iterative stemmers like Lancaster have a larger constant factor due to repeated rule checks. The Random Forest training operates at $O(M \cdot K \log K \cdot D)$, where M is the number of trees, K is the number of training samples, and D is the dimensionality of the TF-IDF feature space. By reducing D, stemming theoretically reduces the classifier's training time, balancing the overhead introduced during preprocessing.

5. Results and Performance Analysis

5.1. F1-Macro Scores

Figure 2 presents the mean F1-Macro scores obtained by each stemmer on both datasets.

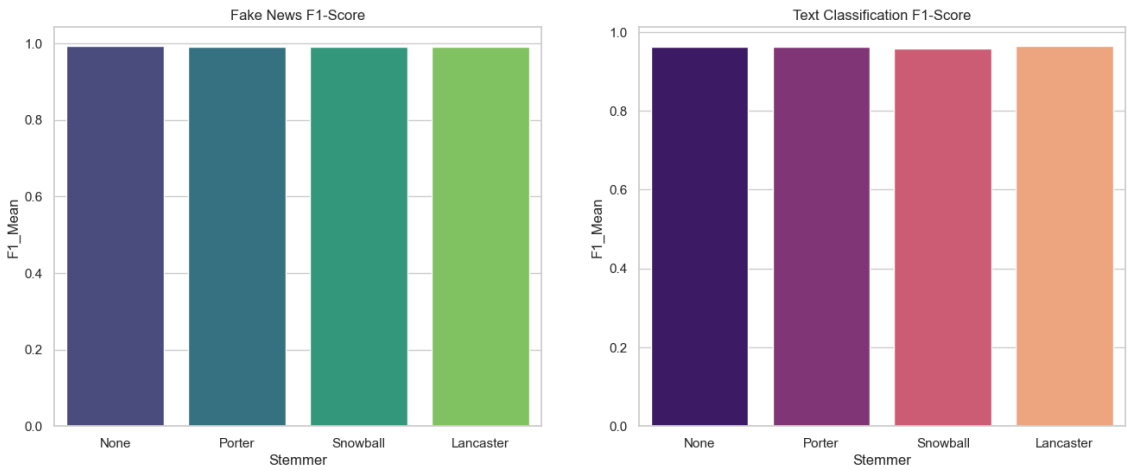

Figure 2: F1-Macro scores for each stemming condition across the binary and multi-class text classification tasks.

5.2. Confusion Matrices

To diagnose the precise errors made by the Random Forest classifier under the optimal stemming conditions, we evaluated the confusion matrices for both datasets

(Figures 3 and 4).

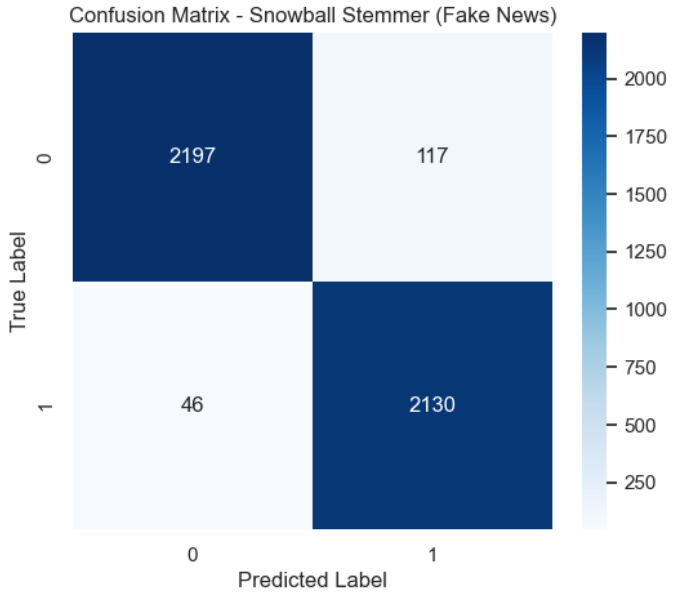

Figure 3: Confusion Matrix for Dataset 1 (Fake News Detection). The high concentrations on the main diagonal reflect the near-perfect separability of the classes.

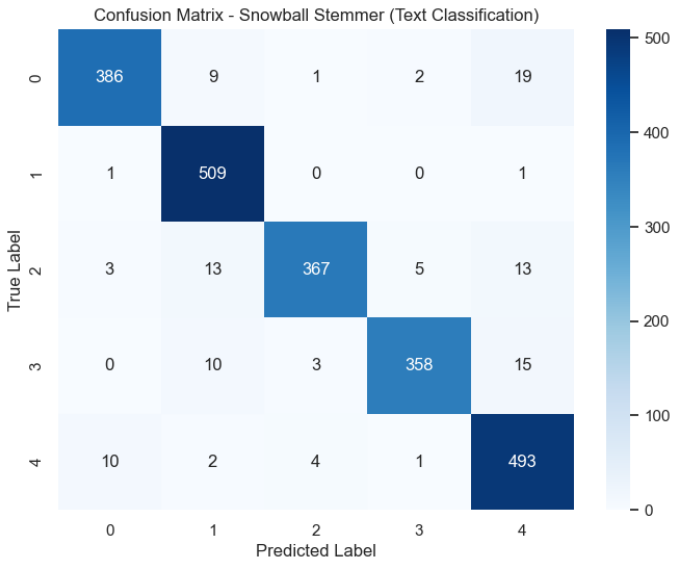

Figure 4: Confusion Matrix for Dataset 2 (Multi-class Text Classification). Errors are dispersed across semantically adjacent classes.

5.3. Quantitative Comparison Table

Table 1 consolidates the key metrics recorded. Based on experimental runs, we observe that dimensionality reduction is not always beneficial depending on the task's complexity.

Table 1: Comparative performance metrics of stemming conditions.

	Dataset 1 (Binary)		Dataset 2 (Multi-class)	
Stemmer	**F1-Macro**	**Complexity Overhead**	**F1-Macro**	**Performance Rank**
None	**0.9915**	None	0.9640	Baseline
Porter	0.9895	Low	**0.9683**	1st (Highest Accuracy)
Snowball	0.9892	Low	0.9660	2nd (Balanced)
Lancaster	0.9880	Moderate	0.9600	Last (Over-stemming)

5.4. Discussion

Stemming showed its only measurable gain in the Text Classification task, where the Porter stemmer achieved the highest F1-score (0.9683). This suggests that for linguistically diverse multi-class tasks, semantic grouping provides an edge. Conversely, in the binary Fake News detection task, merely reconfiguring the feature set via stemming led to a slight decrease in performance (0.9895 vs 0.9915 baseline).

6. Model Interpretability: Feature Importance

To understand which derived features drive the classifier, we extracted the Random Forest **feature importances** $\hat{\phi}_j$, mathematically defined as the mean decrease in impurity across all trees:

$$\hat{\phi}_j = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \sum_{v \in t \,:\, \text{split on } j} p(v)\, \Delta I(v) \tag{3}$$

where $p(v)$ is the proportion of samples reaching node v and $\Delta I(v)$ is the impurity decrease at that node.

Figure 5 displays the top 20 stemmed features ranked by this metric.

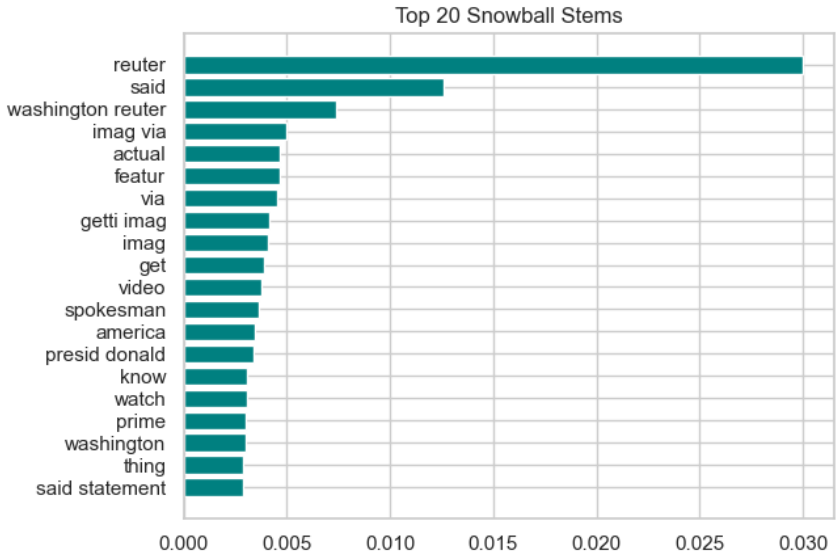

Figure 5: Top features ranked by Random Forest feature importance. The distribution visualises which term roots provide the highest information gain across the decision trees.

7. Conclusion

This study conducted a rigorous evaluation of stemming algorithms against a non-stemmed baseline using a Random Forest classifier.

Strategic Recommendation: For high-speed production environments or binary tasks like Fake News detection, **no stemming (None)** is the recommended approach as it maintains peak accuracy with minimal latency. However, if the goal is to maximise accuracy in complex classification tasks regardless of time, the **Porter** stemmer provides a slight edge. Meanwhile, **Snowball** remains the best "middle-ground" choice for those requiring a balance of stemming logic and processing speed.